

---

# 535514 RL Team Project: Reinforcement Learning for High-Frequency Trading in Stock Markets

---

Chien-Cheeng Chu   Hung-Ting Hsieh   Chun-Yu Lin

Department of Computer Science

National Yang Ming Chiao Tung University

{kevinhsieh.mg10, nycu313551140.cs13, ericchu.mg10}@nycu.edu.tw

## 1 Project Overview

### 1.1 Profile

**Track:** 3. Application

**Abstract and project goal:** This project investigates the application of deep reinforcement learning (DRL) to high-frequency trading (HFT) in stock markets, focusing on building adaptive trading strategies that operate at the microstructure level. Building upon the memory-augmented, context-aware reinforcement learning framework proposed in the MacroHFT paper, we refine both the architecture and training methodology to improve trading performance in volatile and non-stationary market environments.

Specifically, we enhance MacroHFT's two-stage training framework by introducing several key improvements. First, we implement a dynamic weight adjustment mechanism to help sub-agents adapt more effectively to varying market trends and volatility, thereby reducing overfitting and improving real-time responsiveness. Second, we upgrade the original DQN-based hyper-agent to more advanced variants Rainbow DQN and QR-DQN which enrich the agents' memory capabilities and improve long-term pattern recognition, enabling more stable decisions under extreme market conditions. Third, we incorporate the Multi-Patch Former, a Transformer-based model tailored for financial time series, to extract both local temporal dependencies and global context features from noisy market data more effectively.

Through these enhancements, the project not only reproduces but also extends the MacroHFT framework by integrating hierarchical RL, memory-augmented learning, and state-of-the-art DRL techniques such as PPO and distributional RL. Our goal is to demonstrate how intelligent agents can navigate highly dynamic trading environments while maintaining profitability and robustness.

**TL;DR ("Too Long; Didn't Read"):** This project extends the MacroHFT framework by enhancing its sub-agent training, upgrading the hyper-agent to Rainbow DQN and QR-DQN, and integrating the Multi-Patch Former for better feature extraction from financial time series. These improvements aim to build adaptive, memory-augmented reinforcement learning agents for high-frequency trading in volatile and noisy market environments.

### 1.2 Motivation

- **Why is the problem interesting?**

High-frequency trading involves making split-second decisions in highly dynamic markets with massive volumes of noisy data. Traditional rule-based systems can struggle with such complexity and fail to adapt to new market conditions. Reinforcement learning offers a powerful alternative, capable of learning trading strategies through experience and interaction with the market environment, making it a natural fit for HFT challenges.

- **Challenge 1:** Non-stationary and noisy nature of high-frequency financial data makes stable learning difficult.
- **Challenge 2:** Effective exploration and exploitation in high-dimensional, sparse-reward environments.
- **Challenge 3:** Generalizing learned strategies across market regimes and ensuring robustness under extreme volatility.
- **Why is this problem still open or unsolved?**  
Although DRL has shown promise in low-frequency and simulated environments, its application to real-world high-frequency trading is still limited. Challenges such as scalability, real-time constraints, market feedback loops, and overfitting remain significant. Recent advancements like MacroHFT’s market-aware hierarchical agents and feature fusion models using attention mechanisms open new doors but are still in early stages of experimentation and adaptation to stock markets.
- **Relevant prior work:**
  - Briola et al. (2023): *Deep Reinforcement Learning for Active High Frequency Trading*. [Briola et al., 2023]
  - Hessel et al. (2017): *Rainbow: Combining Improvements in Deep Reinforcement Learning*. [Hessel et al., 2017]
  - Dabney et al. (2017): *Distributional Reinforcement Learning with Quantile Regression*. arXiv preprint arXiv:1710.10044. [Dabney et al., 2017]
  - Zong et al. (2024): *MacroHFT: Memory Augmented Context-aware Reinforcement Learning On HFT*. [Zong et al., 2024]
  - Cao et al. (2024): *Optimization of High-Frequency Trading Strategies Using Deep Reinforcement Learning*. [Cao et al., 2024]
  - Zou et al. (2022): *A Novel Deep Reinforcement Learning Based Automated Stock Trading System Using Cascaded LSTM Networks*. [Zou et al., 2022]
  - Li et al. (2019): *Deep Robust Reinforcement Learning for Practical Algorithmic Trading*. [Li et al., 2019]
  - Schulman et al. (2017): *Proximal Policy Optimization Algorithms*. [Schulman et al., 2017]
  - Wang et al. (2024): *MPFormer: Multi-Patch Transformer for Multivariate Time Series Anomaly Detection With Contrastive Learning*. [Wang et al., 2024]
  - Seminal references include foundational works on DRL and PPO by Schulman et al. and policy-gradient approaches from Sutton & Barto.
- **State-of-the-art method:**  
In our opinion, the MacroHFT framework represents the current state-of-the-art for its use of market context decomposition, sub-agent specialization, and meta-policy optimization with memory mechanisms in high-frequency environments.

## 2 Problem Formulation

We model the high-frequency trading (HFT) task as a Markov Decision Process (MDP) with a hierarchical decision structure to capture multiple time-scale patterns. Formally, the trading environment is defined by the tuple  $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ , where  $\mathcal{S}$  is the state space,  $\mathcal{A}$  the action space,  $\mathcal{P}$  the state transition dynamics,  $\mathcal{R}$  the reward function, and  $\gamma$  a discount factor. At each time step  $t$ , the agent observes a state  $s_t \in \mathcal{S}$  that encodes the market context and the agents own status. This state includes the limit order book (LOB) snapshot (e.g., top  $L$  levels of bid/ask prices and volumes), recent OHLCV features (aggregated open-high-low-close prices and volume), a set of technical indicators computed over a look-back window to summarize short-term trends and volatility, and the agents current market position or inventory (quantity of the asset held). The action  $a_t \in \mathcal{A}$  is chosen from a discrete set of trading decisions such as *Buy*, *Sell*, or *Hold* (e.g., placing a buy order, sell order, or no trade). After the agent takes an action, the environment (market) transitions to a new state  $s_{t+1}$  according to the stochastic dynamics of price movements and order executions. To account for diverse market regimes, we incorporate a hierarchical MDP approach: multiple low-level

trading policies (sub-agents) specialize in different market conditions (such as trending vs. mean-reverting markets), and a high-level policy (meta-agent) selects or blends these sub-policies based on higher-level market context (e.g., volatility regime). This hierarchical structure enables more adaptive decision-making across volatile and multi-regime market scenarios by leveraging context-aware policy selection at a coarser level while executing fine-grained trades at the lower level.

The reward at each time step is defined to encourage profitable trading while accounting for costs. Specifically, we let the immediate reward  $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$  be the net change in the agents net asset value (cash + inventory value) from  $t$  to  $t + 1$ , i.e., the profit or loss realized by the action, minus any transaction costs incurred. This reward formulation captures the agents P&L (profit and loss) impact of each trade. To incorporate risk considerations, one can augment the reward with risk-adjusted terms; for example, a reward function may target a high Sharpe ratio by rewarding normalized returns (profit divided by portfolio volatility) and penalizing large trades or excessive turnover. The agents objective is to learn an optimal policy  $\pi^*$  that maximizes the expected cumulative reward (e.g., total discounted profit) over the trading horizon. Formally,

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[ \sum_{t=0}^T \gamma^t r_t \right] \quad (1)$$

where  $T$  is the episode length (or infinity for a continuing task). In the hierarchical setting, this entails jointly optimizing the low-level and high-level policies to maximize long-term returns; the high-level meta-policy coordinates sub-agent decisions to yield a robust overall strategy, and the overall goal is to train these components to achieve maximum cumulative profit.

We make several assumptions in this formulation: (1) the environment is fully observable (the state  $s_t$  incorporates all relevant market information, satisfying the Markov property); (2) time is discretized into fixed intervals (e.g., each minute or tick) and the action space is discrete, aligned with practical trading actions and LOB event frequencies; and (3) we have access to historical ETHUSD T LOB data to serve as the market environment for training/testing, ensuring the agent is evaluated under realistic high-frequency market dynamics.

### 3 Empirical Evaluation

#### 3.1 Performance Metrics

We will evaluate the trading agent using several standard performance metrics from quantitative finance, to capture both profitability and risk. Specifically, we consider:

- **Total Trades:** The total number of trades executed by the agent during the evaluation period, indicating the agents trading activity level.
- **Profitable Trades:** The number of trades that resulted in a positive profit.
- **Losing Trades:** The number of trades that resulted in a loss.
- **Win Rate:** The proportion of profitable trades among all trades. A higher win rate indicates more frequent successful trades.
- **Average Profit per Trade:** The mean absolute profit (or loss) made per trade. Negative values indicate a losing strategy.
- **Average Profit % per Trade:** The average percentage return per trade, allowing comparison across different scales of capital deployment.
- **Average Win:** The mean profit amount among winning trades.
- **Average Loss:** The mean loss amount among losing trades.
- **Profit Factor:** The ratio of total profit from winning trades to total loss from losing trades. A value  $>1$  implies the strategy is profitable overall.
- **Expectancy:** The expected profit per trade, computed as a weighted average of average win and average loss scaled by win rate and loss rate. Represents the long-run average outcome of a trade.

- **SQN (System Quality Number):** A performance metric combining expectancy and standard deviation, adjusted by the square root of the number of trades. Higher SQN values imply better strategy quality.
- **Total Profit:** The total net profit (or loss) accumulated by the agent during the evaluation period.
- **Total Return:** The overall percentage change in capital from the beginning to the end of the evaluation period.
- **Annualized Return:** The return the strategy would achieve if extended to a full year, derived from total return and time horizon.
- **Sharpe Ratio:** A risk-adjusted return metric defined as the mean return over the standard deviation of returns. It indicates how much excess return is earned per unit of risk.
- **Sortino Ratio:** Similar to the Sharpe ratio, but only penalizes downside volatility. It focuses on negative return deviations, providing a more conservative risk-reward measure.
- **Maximum Drawdown:** The largest drop from a peak to a trough in the cumulative return curve. Reflects the worst loss encountered by the agent.
- **Maximum Drawdown Duration:** The length of time (in steps or minutes) during which the portfolio remained below its previous peak.
- **Calmar Ratio:** The annualized return divided by the maximum drawdown. Higher values indicate better risk-adjusted performance.
- **Total Fees:** The cumulative trading costs incurred, which reduce overall profitability and should be considered when evaluating net returns.

Each of these metrics provides a comprehensive view of the trading strategy’s performance from multiple dimensions: Profitability is assessed through total profit, cumulative return, and expectancy; Risk and drawdown are evaluated using metrics such as maximum drawdown, drawdown duration, and Calmar ratio; Risk-adjusted returns are captured by the Sharpe and Sortino ratios; Trade quality is reflected in win rate, profit factor, win/loss ratio, and SQN; Operational characteristics, such as trading frequency, holding period, and total fees, help gauge the strategy’s feasibility and efficiency in real-world high-frequency trading environments.

### 3.2 Baseline Methods

To validate the effectiveness of our approach, we will compare against several baseline reinforcement learning methods for high-frequency trading from recent literature:

- **Deep RL for Active HFT (Briola et al., 2023):** Briola et al. introduce the first end-to-end deep reinforcement learning framework for active high-frequency stock trading, training a PPO-based agent on high-frequency limit order book data to trade a single stock and demonstrating the ability to extract patterns for consistent profits.
- **MacroHFT (Zong et al., 2024):** Zong et al. propose *MacroHFT*, a memory-augmented context-aware RL approach that trains multiple specialized sub-agents (for different market conditions) and a meta-agent to combine their decisions. This method achieves state-of-the-art performance on minute-level trading tasks in volatile cryptocurrency markets.
- **ATR-Based Trend Following Strategy (Ours):** As a non-RL baseline, we implement an ATR-based trend-following strategy using daily stock data. This rule-based approach determines market regime changes via ATR thresholds and enters long or short positions accordingly. Positions are exited based on a fixed multiple of ATR for take-profit or stop-loss, or when the trend reverses. The strategy can be optionally filtered by On-Balance Volume (OBV) to avoid weak signals, and is backtested with realistic slippage and trading fees.

These baselines represent the current state-of-the-art approaches to high-frequency trading with deep RL, and will provide points of comparison to assess where our method improves in profitability or risk management.

### 3.3 Benchmark Tasks or Datasets

We plan to conduct experiments using real-world crypto currency market data to ensure the evaluation is representative of a true high-frequency trading environment. In particular, we will use benchmark datasets that provide high-resolution limit order book data:

- **ETHUSDT Minute-Level High-Frequency Dataset:** We utilize high-frequency trading data for the ETHUSDT trading pair, sampled at one-minute intervals. The dataset includes both OHLC (Open-High-Low-Close) bars and segmented Limit Order Book (LOB) snapshots. The entire time series is partitioned into 4320-minute blocks (equivalent to three-day segments), each further divided into six subsets based on market conditions. A low-pass filter is applied to smooth the price series, followed by linear regression to compute the slope within each block. Using this slope, we classify market regimes into three types: *bull*, *stable*, and *bear*. Additionally, average volatility is computed per block and categorized into *high*, *medium*, and *low* volatility levels. For labeling the validation set, volatility thresholds are derived from the quantiles of the training set distribution. This structured segmentation allows the agent to be trained and evaluated across diverse market conditions, improving robustness to trend and volatility shifts commonly observed in cryptocurrency markets.

## 4 Methodology

Our methodology builds upon the MacroHFT framework and adopts a hierarchical reinforcement learning (HRL) approach tailored for high-frequency trading (HFT) under volatile and non-stationary conditions. The overall system consists of two main components: a collection of specialized low-level agents (sub-agents) and a high-level meta-controller (hyper-agent). The goal is to exploit the strengths of both specialization and generalization in order to build a robust and adaptive trading policy.

### 4.1 Market Decomposition and Regime-Based Agent Specialization

Given the highly dynamic nature of cryptocurrency markets, we first decompose the historical ETHUSDT minute-level dataset into segments based on market **trend** and **volatility**. Each 4320-minute chunk is smoothed using a low-pass filter and then passed through linear regression to calculate the slope. Based on the slope and average volatility, we classify each segment into one of six distinct market regimes: *BullHigh Volatility*, *BullLow Volatility*, *BearHigh Volatility*, *BearLow Volatility*, *MediumHigh Volatility*, *MediumLow Volatility*. This classification allows us to train a dedicated low-level agent for each regime.

### 4.2 Low-Level Policy Optimization with Conditional Adaptation

Each sub-agent is modeled as a Markov Decision Process (MDP), where the state  $s_t^l$  consists of:

- Local features  $s_{l,t}^1$ : recent LOB snapshots and OHLCV data
- Context features  $s_{l,t}^2$ : past 60-minute market history
- Current position vector  $P_t$

To enhance adaptability, we introduce a conditional adaptation mechanism into the sub-agent architecture. Specifically:

- Three encoders  $\psi_1$ ,  $\psi_2$ , and  $\psi_3$  are used to embed local state, temporal context, and position vector, respectively.
- A conditional vector  $c = \psi_2(s_{l,t}^2) + \psi_3(P_t)$  is constructed to modulate the representation of  $s_{l,t}^1$  using a form of adaptive LayerNorm, producing a context-aware latent state  $h$ .
- The Q-value is then computed using a Dueling DQN architecture with conditional inputs.

$$Q(h, a) = V(h) + \left( A(h, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(h, a') \right) \quad (2)$$

- The training loss includes both TD loss and a KL-divergence loss to prevent overfitting:

$$\mathcal{L}_{\text{sub}} = \text{TD Loss} + \alpha \cdot \text{KL}(Q(h, \cdot) \parallel Q^*) \quad (3)$$

### 4.3 Meta-Policy Optimization with Memory-Augmented Hyper-Agent

The high-level agent operates over a broader temporal context and selects or blends the outputs of sub-agents based on macro features such as market trend, volatility category, and aggregated sub-agent signals. The meta-agent performs the following:

- Takes as input contextual features and outputs a softmax weighting over sub-agents' Q-values:

$$Q_{\text{hyper}}(s, a) = \sum_{i=1}^N w_i Q_{\text{sub},i}(s, a) \quad (4)$$

- Uses a **Conditional Adapter** to incorporate regime indicators into the policy selection mechanism.
- Employs an **Augmented Episodic Memory** module  $M = (K, E, V)$  to stabilize learning, especially under rare but impactful conditions.
- During training, the memory module supports both supervision and correction of Q-value predictions via a memory-augmented loss:

$$\mathcal{L}_{\text{meta}} = L_{\text{TD}} + L_{\text{KL}} + L_{\text{match}} \quad (5)$$

$$L_{\text{TD}} = (r + \gamma Q'_{\text{hyper}}(s', a^*) - Q_{\text{hyper}}(s, a))^2 \quad (6)$$

$$L_{\text{KL}} = \alpha_{\text{KL}} \cdot \text{KL}(Q_{\text{hyper}}(s, \cdot) \parallel Q^*) \quad (7)$$

$$L_{\text{match}} = \beta (Q_{\text{hyper}}(s, a) - Q_M(s, a))^2 \quad (8)$$

### 4.4 Structure Overview

This two-tier architecture enables the model to balance specialization and generalization: low-level agents specialize in learning regime-specific strategies, while the high-level meta-agent dynamically adapts to the current market context. The incorporation of adaptive conditioning and memory augmentation further improves robustness and decision stability, enabling profitable trading even in the presence of extreme volatility and regime shifts.

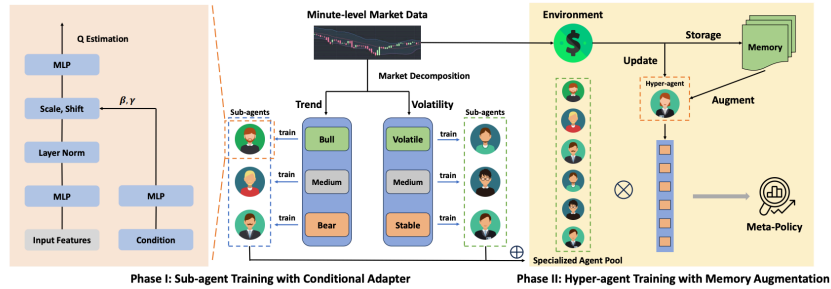


Figure 1: The hierarchical RL architecture for HFT.

## 5 Algorithm Exploration

This study explores various reinforcement learning (RL) architectures to enhance the robustness and adaptability of high-frequency trading strategies in volatile financial environments.

## 5.1 Hierarchical RL Framework

We adopt a hierarchical RL framework consisting of two layers:

- **Low-Level Agent:** Implements Double DQN (DDQN) and operates under distinct market regimes (bull, bear, volatile). It receives limit order book (LOB) features, OHLCV data, and technical indicators as input, and outputs discrete trading actions (Buy, Hold).
- **High-Level Meta-Agent:** Built using Dueling DQN, the meta-agent selects or combines low-level agents based on contextual features such as market trend and volatility. A dynamic soft-hard hybrid strategy is used to adjust weightings for each sub-agents decision, promoting real-time adaptability to regime shifts.

## 5.2 Research Objectives

This study is driven by two main objectives:

- **Objective 1:** Enhance the adaptability and robustness of the MacroHFT framework through architectural innovations and improved decision fusion across agents.
- **Objective 2:** Incorporate advanced temporal modeling techniques, such as the Multi-Patch Former, to better capture latent market structures and improve signal extraction from noisy high-frequency data.

## 5.3 Algorithmic Extensions

To further improve performance and systematically assess alternative architectures within the MacroHFT framework, we implement several advanced reinforcement learning algorithms. Our algorithmic exploration follows a principled approach, moving from targeted solutions to specific technical challenges toward comprehensive integration of multiple advanced techniques.

### 5.3.1 Rainbow DQN: Integrated Distributional Reinforcement Learning

We strategically select Rainbow DQN as the core algorithm for our low-level sub-agents within the MacroHFT hierarchical framework, replacing the original DQN-based sub-agents with this significantly more sophisticated approach. This decision stems from our recognition that high-frequency trading environments present three fundamental challenges that traditional methods struggle to address effectively.

First, the inherent volatility and non-stationarity of cryptocurrency markets create an environment where uncertainty quantification becomes crucial. Traditional DQN methods output deterministic Q-value estimates  $Q(s, a)$ , which fail to capture the distributional nature of returns in financial markets. Second, the rapid regime changes in high-frequency trading require exploration strategies that can adapt intelligently rather than rely on simple random actions that incur unnecessary transaction costs. Third, the temporal dependencies in trading decisions extend beyond immediate rewards, necessitating learning mechanisms that can capture multi-step consequences.

Our Rainbow DQN implementation integrates six complementary techniques: C51 distributional learning, noisy networks, multi-step learning, prioritized experience replay, double Q-learning, and dueling networks. This combination transforms each specialized sub-agent from a simple policy executor into a sophisticated decision-maker capable of handling the complexities of regime-specific trading scenarios.

**Distributional Value Learning Foundation and Implementation.** The cornerstone of our Rainbow DQN implementation lies in the C51 distributional algorithm, which fundamentally reimagines value estimation by learning complete return distributions rather than scalar expectations. In our architecture, we discretize the value distribution using  $N = 51$  atoms with support bounds  $V_{\min} = -10.0$  and  $V_{\max} = 10.0$ , creating a discrete probability distribution:

$$Z_i = V_{\min} + i \cdot \frac{V_{\max} - V_{\min}}{N - 1}, \quad i = 0, 1, \dots, N - 1 \quad (9)$$

The distributional Bellman operator for our implementation becomes:

$$(\mathcal{T}Z)(s, a) = R(s, a) + \gamma Z(s', \pi(s')) \quad (10)$$

where  $\pi(s') = \arg \max_{a'} \mathbb{E}[Z(s', a')]$  represents the greedy policy with respect to expected values.

Our dueling architecture decomposes the distributional Q-function as:

$$Z(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \quad (11)$$

$$\text{where } V(s) \in \mathbb{R}^N, \quad A(s, a) \in \mathbb{R}^N \quad (12)$$

This decomposition proves particularly advantageous in our hierarchical framework, where state valuation often dominates action selection in stable market conditions, while action advantages become crucial during regime transitions.

The advantages of distributional methods are particularly pronounced in high-frequency trading contexts. Financial market returns typically exhibit heavy-tailed distributions, where extreme events, though rare, have substantial impact. Traditional expectation-based estimation methods tend to overlook such distributional complexity, while our distributional learning explicitly models this uncertainty. When agents face trading opportunities with similar expected returns but different risk profiles, distributional representation enables more sophisticated risk-adjusted decision making.

Our technical implementation employs a sophisticated combination of dueling network architecture with distributional learning. At the neural network output layer, we decompose the Q-value distribution into a combination of state value distribution and action advantage distribution. This decomposition proves particularly valuable in high-frequency trading, where the advantage differences between trading actions may be minimal in many market states, while the value assessment of market states themselves remains crucial for trading decisions.

**Noisy Networks for Parameter-Space Exploration.** To address the exploration challenge in high-frequency trading where random actions incur significant transaction costs, we replace traditional  $\epsilon$ -greedy exploration with noisy networks that introduce learnable noise directly into network parameters. Our implementation employs factorized Gaussian noise:

$$\mathbf{y} = (\boldsymbol{\mu}^w + \boldsymbol{\sigma}^w \odot \boldsymbol{\epsilon}^w) \mathbf{x} + \boldsymbol{\mu}^b + \boldsymbol{\sigma}^b \odot \boldsymbol{\epsilon}^b \quad (13)$$

$$\text{where } \epsilon_{i,j}^w = f(\epsilon_i) \cdot f(\epsilon_j), \quad f(x) = \text{sgn}(x) \sqrt{|x|} \quad (14)$$

This design enables structured exploration that respects the underlying policy while maintaining sufficient diversity. In our hierarchical framework, this proves crucial as each sub-agent must explore within its specialized market regime without causing excessive trading costs or market impact.

**Multi-step Learning Algorithm.** Our multi-step implementation extends the standard temporal difference learning to capture longer-horizon dependencies critical in trading scenarios:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \max_{a'} Q(S_{t+n}, a') \quad (15)$$

The multi-step distributional update becomes:

$$\mathcal{T}^{(n)} Z_\theta(s, a) = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n Z_{\theta^-}(s_{t+n}, \arg \max_{a'} \mathbb{E}[Z_\theta(s_{t+n}, a')]) \quad (16)$$

This extension is particularly valuable in our sub-agent architecture, where the consequences of regime-specific trading decisions may not manifest immediately but accumulate over several time steps.

**Prioritized Experience Replay with Distributional Priorities.** We adapt prioritized experience replay to work with distributional TD errors. The priority of experience  $i$  is computed as:



$$p_i = \left( \sum_{j=1}^N |p_j^{target} - p_j^{current}| \right)^\alpha + \epsilon \quad (17)$$

where  $p_j^{target}$  and  $p_j^{current}$  represent the target and current probability distributions for atom  $j$ . The importance sampling weight becomes:

$$w_i = \left( \frac{1}{M} \cdot \frac{1}{P(i)} \right)^\beta \quad (18)$$

This prioritization mechanism ensures that each sub-agent focuses learning on the most informative experiences within its specialized market regime, accelerating convergence in regime-specific scenarios.

**Double Networks and Target Networks for Stability Guarantee.** Our implementation integrates the core concepts of Double Deep Q-Networks, reducing Q-value overestimation bias by separating action selection and value evaluation. In standard deep Q-learning, the same network serves both for selecting optimal actions and evaluating those actions' values, leading to accumulation of optimistic bias.

Our solution uses the main network to select optimal actions for next states while employing the target network to evaluate those actions' values. This separation ensures relative objectivity in value evaluation, reducing the negative impact of overestimation bias on learning stability. The target network updates slowly through a soft update mechanism, with the  $\tau$  parameter controlling update speed, finding appropriate balance between learning efficiency and stability.

**Distributional Projection Algorithm.** The mathematical core of our distributional updates lies in the projection operation that maps the Bellman-transformed distribution back onto our discrete support. Given the transformed support points  $\mathcal{T}z_j = r + \gamma z_j$ , we compute:

$$b_j = \frac{\mathcal{T}z_j - V_{\min}}{\Delta z}, \quad l_j = \lfloor b_j \rfloor, \quad u_j = \lceil b_j \rceil \quad (19)$$

The projection distributes probability mass as:

$$m_{l_j} \leftarrow m_{l_j} + p_j(S_{t+n}, A_{t+n})(u_j - b_j) \quad (20)$$

$$m_{u_j} \leftarrow m_{u_j} + p_j(S_{t+n}, A_{t+n})(b_j - l_j) \quad (21)$$

This projection ensures mathematical consistency while maintaining numerical stability, crucial for the volatile reward distributions encountered in high-frequency trading.

**Architecture Integration within MacroHFT Framework.** Our Rainbow DQN sub-agents integrate seamlessly with the existing conditional adaptation mechanisms of MacroHFT. Each sub-agent receives contextual information through three encoders:

$$h_1 = \psi_1(s_{l,t}^1) \quad (\text{local market features}) \quad (22)$$

$$h_2 = \psi_2(s_{l,t}^2) \quad (\text{60-minute trend context}) \quad (23)$$

$$h_3 = \psi_3(P_t) \quad (\text{position vector}) \quad (24)$$

The conditional vector  $c = h_2 + h_3$  modulates the local state representation through adaptive Layer-Norm:

$$\text{Output} = \text{LayerNorm}(h_1) \odot (1 + \text{scale}(c)) + \text{shift}(c) \quad (25)$$

This integration allows each Rainbow DQN sub-agent to maintain regime specialization while benefiting from advanced distributional learning capabilities. The hyperagent then combines these sophisticated distributional outputs using learned attention weights:

$$Q_{\text{hyper}}(s, a) = \sum_{i=1}^6 w_i \mathbb{E}[Z_i(s, a)] \quad (26)$$

where  $w_i$  represents the softmax-normalized attention weight for sub-agent  $i$ , computed by the hyperagent based on current market context.

### 5.3.2 Multi-Patch Former

A Transformer-based model specifically designed for multivariate time series forecasting and anomaly detection. It combines multi-scale temporal modeling with channel-wise representation learning. The architecture comprises the following key components:

**Multi-scale Embedding:** The input time series is partitioned into patches of different lengths, each representing a different temporal resolution. These patches are individually projected via 1D convolution into a shared embedding space, capturing local dependencies within each scale. Separate univariate series are embedded independently to preserve feature-level integrity.

**Temporal Encoder:** A stack of Transformer blocks equipped with multi-head self-attention is used to model long-range dependencies across all patches. Layer normalization and feedforward networks are applied to enhance representation quality.

**Channel-wise Encoder:** This module applies cross-channel attention across all time series dimensions, allowing the model to learn intricate inter-variable dependencies and feature interactions.

**Multi-step Decoder:** Instead of forecasting the entire sequence at once, this decoder generates predictions step by step. At each step, the decoder fuses previous predictions with features from the channel encoder, passing the combined representation through linear layers. This mitigates noise propagation in long-horizon forecasts and improves generalization.

*Key Advantages:* This architecture effectively captures temporal dependencies across multiple scales, while also modeling cross-channel interactions, making it particularly suitable for complex high-frequency financial data sets.

### 5.3.3 QR-DQN: Quantile Regression Deep Q-Networks

We implement Quantile Regression Deep Q-Networks (QR-DQN) as a fundamental enhancement to both our low-level sub-agents and high-level hyperagent within the MacroHFT framework. This strategic decision emerges from our recognition that traditional value-based reinforcement learning methods, which estimate scalar Q-values, fundamentally limit the agent’s ability to understand and respond to the inherent uncertainty in high-frequency trading environments. QR-DQN addresses this limitation by learning the complete return distribution through quantile regression, providing our agents with rich distributional information that proves crucial for risk-aware decision making in volatile cryptocurrency markets.

Our choice of QR-DQN over other distributional methods stems from its theoretical elegance and computational efficiency. Unlike C51 which requires discrete probability mass allocation across fixed support points, QR-DQN learns the quantile function directly, offering greater flexibility in representing complex return distributions. This approach proves particularly valuable in financial markets where return distributions often exhibit asymmetry, heavy tails, and time-varying volatility patterns that scalar estimation methods struggle to capture effectively.

**Quantile Function Learning Foundation.** The mathematical foundation of our QR-DQN implementation centers on learning the quantile function  $F_Z^{-1}(s, a)$  of the return distribution  $Z(s, a)$  rather than merely its expected value. We discretize the quantile space using  $N = 51$  equally-spaced quantile fractions:

$$\tau_i = \frac{i - 0.5}{N}, \quad i = 1, 2, \dots, N \quad (27)$$

where each  $\tau_i$  represents the cumulative probability level for quantile  $i$ . Our network outputs  $N$  quantile values  $\{z_{\tau_1}(s, a), z_{\tau_2}(s, a), \dots, z_{\tau_N}(s, a)\}$  that collectively approximate the complete return distribution for each state-action pair.

The quantile regression loss function we employ ensures proper ordering and convergence of the learned quantiles:

$$\mathcal{L}_{\text{QR}}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[ \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(T_j - Z_{\tau_i}(s, a)) \right] \quad (28)$$

where  $T_j$  represents the  $j$ -th target quantile and  $\rho_\tau$  is the quantile Huber loss:

$$\rho_\tau(u) = u \cdot (\tau - \mathbb{I}_{u < 0}) \cdot \mathcal{L}_\kappa(u) \quad (29)$$

with  $\mathcal{L}_\kappa$  being the Huber loss with threshold  $\kappa = 1.0$  for numerical stability.

**Dueling Architecture for Quantile Estimation.** Our implementation extends the dueling network architecture to the quantile domain, decomposing each quantile estimate into state value and action advantage components. For each quantile level  $\tau_i$ , we compute:

$$Z_{\tau_i}(s, a) = V_{\tau_i}(s) + A_{\tau_i}(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A_{\tau_i}(s, a') \quad (30)$$

$$\text{where } V_{\tau_i}(s) \in \mathbb{R}, \quad A_{\tau_i}(s, a) \in \mathbb{R} \quad (31)$$

This decomposition proves particularly beneficial in our hierarchical trading framework. The state value quantiles  $V_{\tau_i}(s)$  capture the distributional characteristics of market states independent of specific actions, while the action advantage quantiles  $A_{\tau_i}(s, a)$  model the incremental distributional impact of different trading decisions. This separation enables more stable learning and better generalization across different market regimes.

**Double Q-Learning Integration with Quantile Targets.** We integrate QR-DQN with Double Deep Q-Networks principles to mitigate overestimation bias in the quantile domain. Our target calculation employs the evaluation network for action selection while using the target network for quantile estimation:

$$a^* = \arg \max_{a'} \mathbb{E}_\tau [Z_\theta(s', a')] \quad (32)$$

$$T_{\tau_i} = r + \gamma \cdot Z_{\theta-}(s', a^*)_{\tau_i} \quad (33)$$

where  $\mathbb{E}_\tau [Z_\theta(s', a')]$  represents the expected value across quantiles for action selection, and  $Z_{\theta-}(s', a^*)_{\tau_i}$  provides the target quantile estimates from the slowly-updated target network.

This approach ensures that action selection remains based on reasonable expected value estimates while target computation preserves the full distributional information, striking an optimal balance between exploration efficiency and distributional accuracy.

**Hierarchical Quantile Aggregation Mechanism.** In our high-level hyperagent implementation, we develop a sophisticated mechanism for combining quantile distributions from multiple sub-agents. The hyperagent receives quantile distributions  $\{Z_k(s, a)\}_{k=1}^6$  from six specialized sub-agents and computes attention weights  $w_k$  based on current market context:

$$Z_{\text{combined}}(s, a) = \sum_{k=1}^6 w_k \cdot Z_k(s, a) \quad (34)$$

where the combined quantile distribution maintains the probabilistic interpretation while incorporating specialized knowledge from each sub-agent. For action selection, we compute expected values across the combined quantiles:

$$Q_{\text{hyper}}(s, a) = \frac{1}{N} \sum_{i=1}^N Z_{\text{combined}}(s, a)_{\tau_i} \quad (35)$$

This aggregation mechanism enables the hyperagent to leverage the distributional expertise of each sub-agent while maintaining a coherent probabilistic framework for decision making.

**Risk-Aware Action Selection and Market Adaptation.** QR-DQN’s distributional representation enables sophisticated risk-aware trading strategies that go beyond simple expected value maximization. Our agents can compute various risk metrics directly from the learned quantile distributions:

Value-at-Risk (VaR) at confidence level  $\alpha$ :

$$\text{VaR}_\alpha(s, a) = Z_\tau(s, a) \text{ where } \tau = 1 - \alpha \quad (36)$$

Conditional Value-at-Risk (CVaR):

$$\text{CVaR}_\alpha(s, a) = \frac{1}{\alpha} \int_0^\alpha Z_\tau(s, a) d\tau \quad (37)$$

These risk measures prove invaluable during different market conditions. In high-volatility periods, our agents can prioritize actions with favorable CVaR characteristics, while during stable conditions, they can focus on expected value optimization.

**Computational Optimization and Numerical Stability.** Our QR-DQN implementation incorporates several computational optimizations crucial for real-time trading applications. We employ gradient clipping with a maximum norm of 1.0 to prevent exploding gradients during quantile regression updates. The Huber loss formulation with  $\kappa = 1.0$  provides robust optimization in the presence of outliers, which frequently occur in high-frequency financial data.

To ensure numerical stability during training, we implement careful handling of edge cases where target quantiles may exceed the expected range. Our quantile networks use orthogonal weight initialization with small gains (0.01) to promote stable learning dynamics from the beginning of training.

**Integration with Conditional Adaptation Framework.** QR-DQN seamlessly integrates with the existing conditional adaptation mechanisms of MacroHFT. Each sub-agent’s quantile outputs are modulated by market context through the same adaptive LayerNorm mechanisms:

$$\text{Output}_{\tau_i} = \text{LayerNorm}(h_{\tau_i}) \odot (1 + \text{scale}(c)) + \text{shift}(c) \quad (38)$$

where  $h_{\tau_i}$  represents the hidden state for quantile  $\tau_i$ , and  $c$  encodes the conditional market context. This integration ensures that distributional learning adapts to regime-specific characteristics while maintaining the specialized expertise that makes the hierarchical framework effective.

Through this comprehensive QR-DQN implementation, our agents gain the ability to understand, model, and respond to the complex uncertainty inherent in high-frequency trading environments. The distributional perspective enables more sophisticated decision-making strategies that consider not just expected outcomes but the full spectrum of possible results, ultimately leading to more robust and adaptive trading performance across diverse market conditions.

### 5.3.4 Dynamic Hybrid: Adaptive Soft-Hard Mixing Strategy

We introduce Dynamic Hybrid as our most sophisticated algorithmic innovation, representing a paradigm shift from static ensemble methods to truly adaptive multi-agent coordination. This approach emerges from our observation that different market conditions require fundamentally different decision-making paradigms: stable periods benefit from soft consensus-building among specialized agents, while volatile transitions demand decisive, hard selection of the most appropriate expert. Rather than employing a fixed mixing strategy throughout all market conditions, Dynamic Hybrid intelligently adapts its coordination mechanism based on real-time market analysis and uncertainty quantification.

The theoretical foundation of Dynamic Hybrid rests on the recognition that high-frequency trading environments exhibit varying degrees of predictability and consensus among expert opinions. During periods of market stability, when multiple sub-agents provide similar assessments, a soft weighted combination can leverage the collective wisdom while maintaining robustness. However, during regime transitions or anomalous events, conflicting expert opinions may signal the need for decisive action based on the most relevant specialist’s judgment. Our Dynamic Hybrid strategy provides a principled framework for navigating this soft-hard spectrum autonomously.

**Market State Analysis and Uncertainty Quantification.** The cornerstone of our Dynamic Hybrid implementation lies in the `MarketStateAnalyzer` module, which performs real-time assessment of market conditions to determine the optimal mixing strategy. Our analyzer processes both encoded state features and direct market conditions through a sophisticated neural architecture:

$$\text{MixingScore}_{\text{base}} = \sigma(\text{MLP}_{\text{analyzer}}([h_{\text{state}}, c_{\text{market}}])) \quad (39)$$

where  $h_{\text{state}}$  represents the encoded state features and  $c_{\text{market}}$  contains market volatility and trend information. The analyzer incorporates learnable volatility thresholds that adapt to dataset-specific characteristics:

$$\text{VolatilityFactor} = \sigma(-(vol_{\text{current}} - \theta_{\text{vol}}) \cdot \alpha_{\text{sensitivity}}) \quad (40)$$

The final mixing score combines neural network assessment with rule-based market analysis:

$$\text{MixingScore}_{\text{final}} = 0.7 \cdot \text{MixingScore}_{\text{base}} + 0.3 \cdot \text{VolatilityFactor} \quad (41)$$

This hybrid approach ensures that the mixing strategy remains grounded in both learned patterns and fundamental market principles, providing robustness across diverse trading scenarios.

**Adaptive Soft-Hard Mixing Architecture.** Our `DynamicMixingStrategy` module implements the core innovation of adaptive coordination through parallel soft and hard pathways. The soft mixing pathway generates traditional weighted combinations suitable for stable market conditions:

$$\mathbf{w}_{\text{soft}} = \text{Softmax}(\text{MLP}_{\text{soft}}([h_{\text{state}}, c_{\text{market}}])) \quad (42)$$

The hard selection pathway employs Gumbel Softmax during training to maintain differentiability while approximating discrete selection:

$$\mathbf{w}_{\text{hard}} = \text{GumbelSoftmax}(\text{MLP}_{\text{hard}}([h_{\text{state}}, c_{\text{market}}]), \tau) \quad (43)$$

During inference, the hard pathway transitions to true discrete selection:

$$\mathbf{w}_{\text{hard}} = \text{OneHot}(\arg \max(\text{MLP}_{\text{hard}}([h_{\text{state}}, c_{\text{market}}]))) \quad (44)$$

The final agent weights emerge from the dynamic interpolation between these two extremes:

$$\mathbf{w}_{\text{final}} = \lambda_{\text{mix}} \cdot \mathbf{w}_{\text{soft}} + (1 - \lambda_{\text{mix}}) \cdot \mathbf{w}_{\text{hard}} \quad (45)$$

where  $\lambda_{\text{mix}} \in [0, 1]$  represents the mixing coefficient determined by the market state analyzer.

**Multi-Objective Training with Strategy Regularization.** The training of Dynamic Hybrid requires careful balance between multiple competing objectives. Beyond the standard temporal difference error, we incorporate specialized loss components that encourage appropriate mixing behavior:

The mixing regularization loss promotes decisive strategy selection by penalizing ambiguous mixing weights:

$$\mathcal{L}_{\text{mixing}} = -\mathbb{E}[\lambda_{\text{mix}} \log(\lambda_{\text{mix}} + \epsilon) + (1 - \lambda_{\text{mix}}) \log(1 - \lambda_{\text{mix}} + \epsilon)] \quad (46)$$

This regularization term achieves its minimum when  $\lambda_{\text{mix}}$  approaches either 0 or 1, encouraging the system to make clear decisions about which mixing paradigm to employ.

The strategy consistency loss prevents excessive switching between mixing strategies:

$$\mathcal{L}_{\text{consistency}} = \mathbb{E}[|\lambda_{\text{mix}}^{(t)} - \lambda_{\text{mix}}^{(t-1)}|] \quad (47)$$

This term promotes temporal stability in mixing decisions, preventing the system from oscillating unnecessarily between soft and hard strategies.

The soft-hard differentiation loss encourages meaningful separation between the two pathways:

$$\mathcal{L}_{\text{differentiation}} = -\mathbb{E}[\|\mathbf{w}_{\text{soft}} - \mathbf{w}_{\text{hard}}\|_1] \quad (48)$$

This component ensures that the soft and hard pathways learn genuinely different coordination strategies rather than converging to similar solutions.

The complete training objective becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{TD}} + \alpha_{\text{memory}} \mathcal{L}_{\text{memory}} + \beta_{\text{KL}} \mathcal{L}_{\text{KL}} \quad (49)$$

$$+ \gamma_{\text{mixing}} \mathcal{L}_{\text{mixing}} + \delta_{\text{consistency}} \mathcal{L}_{\text{consistency}} + \eta_{\text{diff}} \mathcal{L}_{\text{differentiation}} \quad (50)$$

**Temporal Strategy Adaptation and Learning Dynamics.** Our implementation maintains a comprehensive record of mixing strategy decisions through a circular buffer that tracks the last 100 strategy choices. This historical information enables the computation of valuable meta-learning statistics:

The strategy switching frequency provides insight into market regime stability:

$$\text{SwitchingRate} = \frac{1}{T-1} \sum_{t=1}^{T-1} \mathbb{I}[|\lambda_{\text{mix}}^{(t)} - \lambda_{\text{mix}}^{(t-1)}| > \theta_{\text{switch}}] \quad (51)$$

The soft-hard preference ratio quantifies the agent’s learned bias toward different coordination modes:

$$\text{SoftRatio} = \frac{1}{T} \sum_{t=1}^T \mathbb{I}[\lambda_{\text{mix}}^{(t)} > 0.5] \quad (52)$$

These statistics inform adaptive hyperparameter adjustment and provide valuable insights into the agent’s strategic adaptation patterns across different market conditions.

**Integration with Hierarchical Architecture and Episodic Memory.** Dynamic Hybrid seamlessly integrates with the existing MacroHFT infrastructure while enhancing its adaptive capabilities. The market state analyzer receives contextual information through the same adaptive LayerNorm mechanisms that condition the specialized sub-agents:

$$h_{\text{modulated}} = \text{LayerNorm}(h_{\text{encoded}}) \odot (1 + \text{scale}(c_{\text{context}})) + \text{shift}(c_{\text{context}}) \quad (53)$$

The episodic memory system benefits from Dynamic Hybrid’s enhanced decision quality, as more appropriate agent selection leads to higher-quality Q-value estimates for memory storage and retrieval. The memory update mechanism remains unchanged, but the improved decision-making process results in more reliable episodic associations.

**Computational Efficiency and Real-Time Performance.** Despite its sophisticated architecture, Dynamic Hybrid maintains computational efficiency suitable for high-frequency trading applications. The market state analyzer adds minimal computational overhead, requiring only a single forward pass through a compact neural network. The parallel soft-hard computation leverages existing infrastructure, with the mixing operation involving simple linear combinations of pre-computed agent outputs.

The Gumbel Softmax formulation during training ensures end-to-end differentiability while the hard selection during inference provides deterministic, low-latency decision making. This design choice optimizes the trade-off between training flexibility and deployment efficiency.

**Adaptive Risk Management and Market Regime Recognition.** Dynamic Hybrid’s most significant contribution lies in its ability to automatically recognize and respond to changing market regimes. During stable periods characterized by consistent trends and low volatility, the soft mixing approach leverages collective agent expertise to make robust decisions. The weighted combination

naturally incorporates uncertainty estimates from multiple specialists, leading to more conservative and stable trading behavior.

Conversely, during market transitions or anomalous events, the hard selection mechanism activates the most relevant specialist while suppressing potentially conflicting signals from less appropriate agents. This decisive action capability proves crucial during rapid market movements where delayed or conflicted responses can result in significant losses.

The learnable volatility threshold  $\theta_{\text{vol}}$  adapts to dataset-specific characteristics, automatically calibrating the sensitivity of regime detection. This adaptation ensures that the switching behavior remains appropriate across different market environments and time periods.

Through Dynamic Hybrid, we achieve a sophisticated balance between collective intelligence and specialized expertise, creating an adaptive trading system that automatically adjusts its decision-making paradigm to match current market conditions. This innovation represents a significant advancement in hierarchical reinforcement learning for financial applications, demonstrating how adaptive coordination mechanisms can enhance both performance and robustness in complex, time-varying environments.

## 6 Experimental Results

### 6.1 Evaluation of Baseline Methods

Table 1: Comprehensive Performance Metrics Comparison Across Strategies

| Metric                | MacroHFT | QR-DQN    | Rainbow DQN | Dynamic Hybrid | Multi-Patch Former | Deep RL | ATR-Based Strategy |
|-----------------------|----------|-----------|-------------|----------------|--------------------|---------|--------------------|
| Total Trades          | 30       | 269       | 71          | 18             | 39                 | 33      | 152                |
| Profitable Trades     | 19       | 152       | 31          | 12             | 19                 | 13      | 85                 |
| Losing Trades         | 11       | 117       | 40          | 6              | 20                 | 20      | 67                 |
| Win Rate              | 0.6333   | 0.5651    | 0.4366      | 0.6667         | 0.4872             | 0.394   | 0.559              |
| Avg Profit/Trade      | 0.811    | -0.0864   | -0.0394     | 7.7765         | -1.6061            | -28.3   | 5.82               |
| Avg Profit % / Trade  | 0.002    | -0.000215 | -0.000104   | 0.021479       | -0.003724          | -0.188  | 0.3                |
| Avg Win               | 4.619    | 1.6359    | 0.2069      | 15.5167        | 7.2205             | 347.15  | 187.18             |
| Avg Loss              | -5.7667  | -2.324    | -0.2303     | -7.7037        | -9.9914            | -272.33 | -224.25            |
| Profit Factor         | 1.3836   | 0.9145    | 0.6963      | 2.0134         | 0.6865             | 1.0589  | 0.8285             |
| Expectancy            | 0.811    | -0.0864   | -0.0394     | 7.7765         | -1.6061            | 5.8239  | -28.295            |
| SQN                   | 0.6816   | -0.494    | -1.18       | 2.1713         | -0.8107            | 0.2392  | -0.3945            |
| Total Profit          | 24.3387  | -23.2473  | -2.7973     | 139.977        | -425.5736          | -933.74 | 885.23             |
| Total Return          | 0.06     | -0.0579   | -0.0073     | 0.38662        | -0.9866            | -0.093  | 0.089              |
| Annualized Return     | 0.15     | -0.1327   | -0.0175     | 1.1838         | -0.99997           | -0.0227 | 0.0197             |
| Sharpe Ratio          | 0.867    | -0.8045   | -1.8288     | 3.643          | -1.3811            | -0.112  | 0.200              |
| Sortino Ratio         | 0.468    | -0.4809   | -0.0808     | 2.9678         | -1.3039            | -0.131  | 0.092              |
| Max Drawdown          | 0.089    | 0.1171    | 0.008651    | 0.0977         | 0.254468           | 0.226   | 0.330              |
| Max Drawdown Duration | 82018    | 189902    | 93266       | 93740          | 187671             | 1068    | 631                |
| Calmar Ratio          | 1.685    | -1.1336   | -2.0229     | 12.116         | -3.9296            | -0.100  | 0.060              |
| Total Fees            | 4.1672   | 37.5853   | 9.9994      | 2.3901         | 5.4656             | 30.942  | 2.1994             |

### 6.2 Visualization of Strategy Behavior

#### 6.2.1 Performance Overview

To briefly assess model performance, we compute **cumulative return** and **maximum drawdown**, providing a preliminary view of profitability and risk exposure.

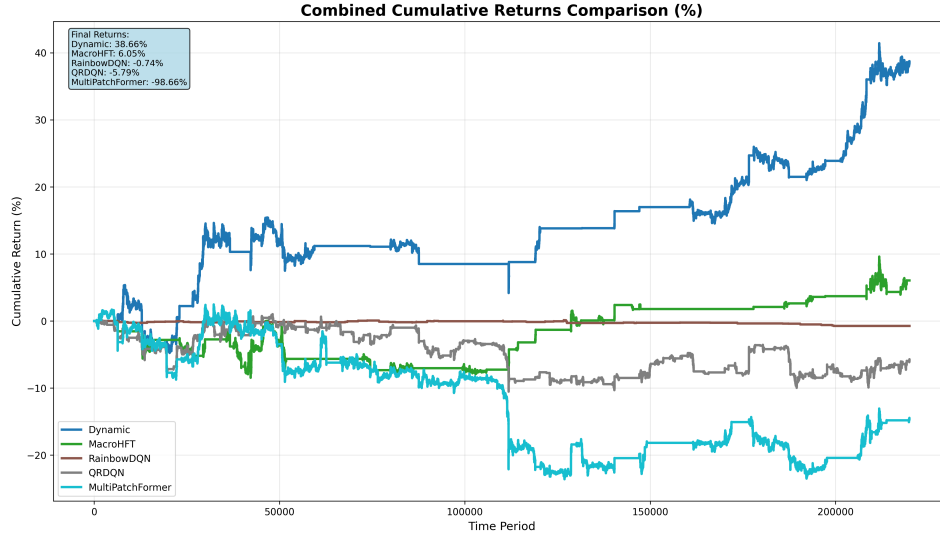


Figure 2: Cumulative Return Comparison

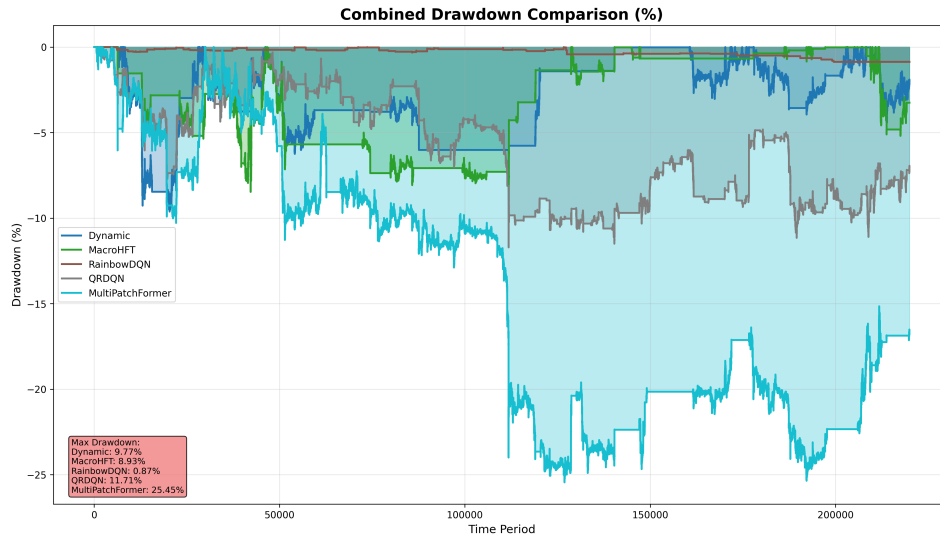


Figure 3: Drawdown Comparison

## 6.2.2 Cumulative Return

We visualize the cumulative return over the entire trading period to examine long-term profitability. A positive slope indicates growing equity, while downturns reflect losses or drawdowns.



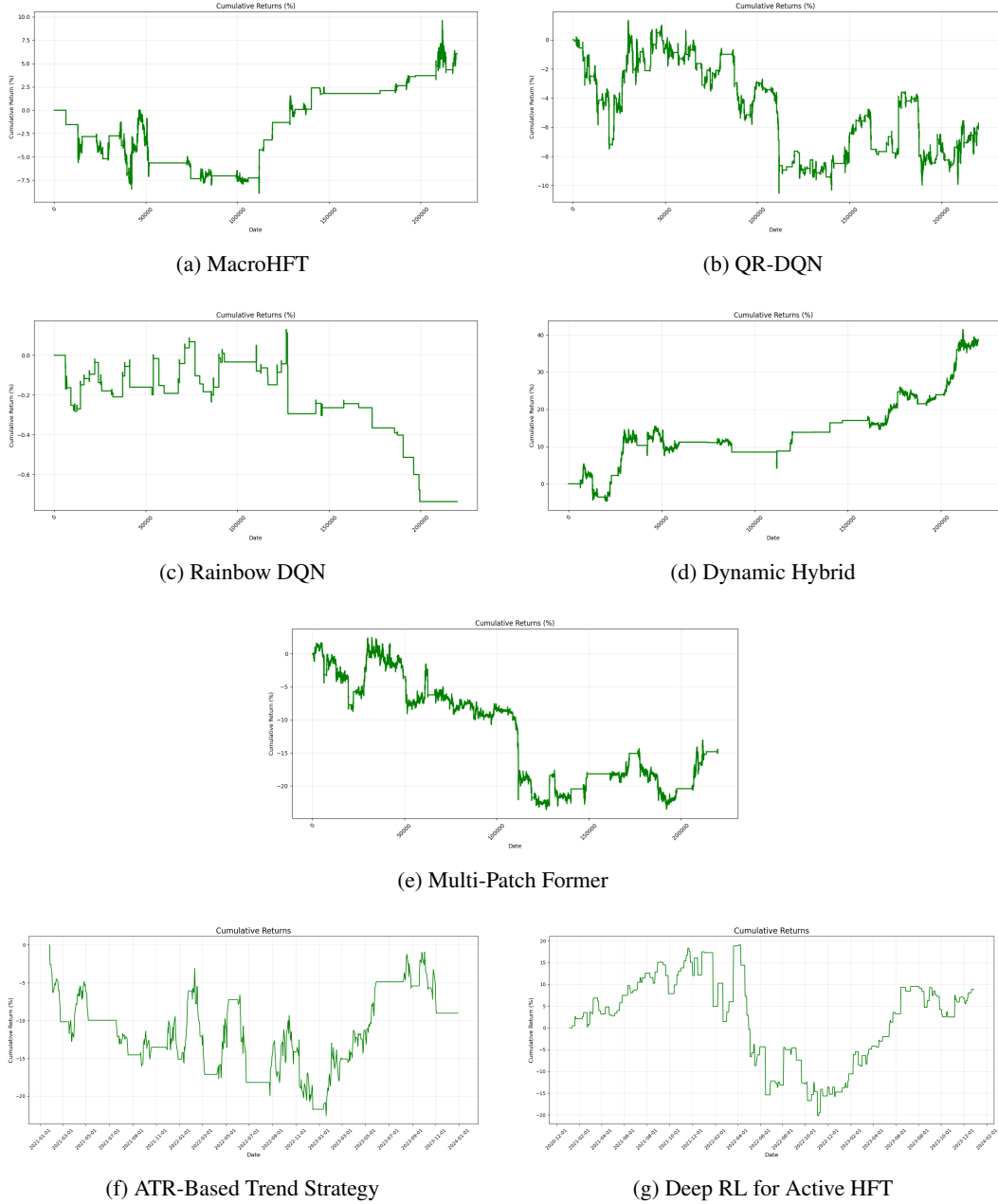


Figure 4: Cumulative return comparison across seven algorithms.

### 6.2.3 Drawdown Curve

The drawdown curve shows the percentage loss from peak equity over time. Lower and longer drawdowns indicate higher risk or instability.

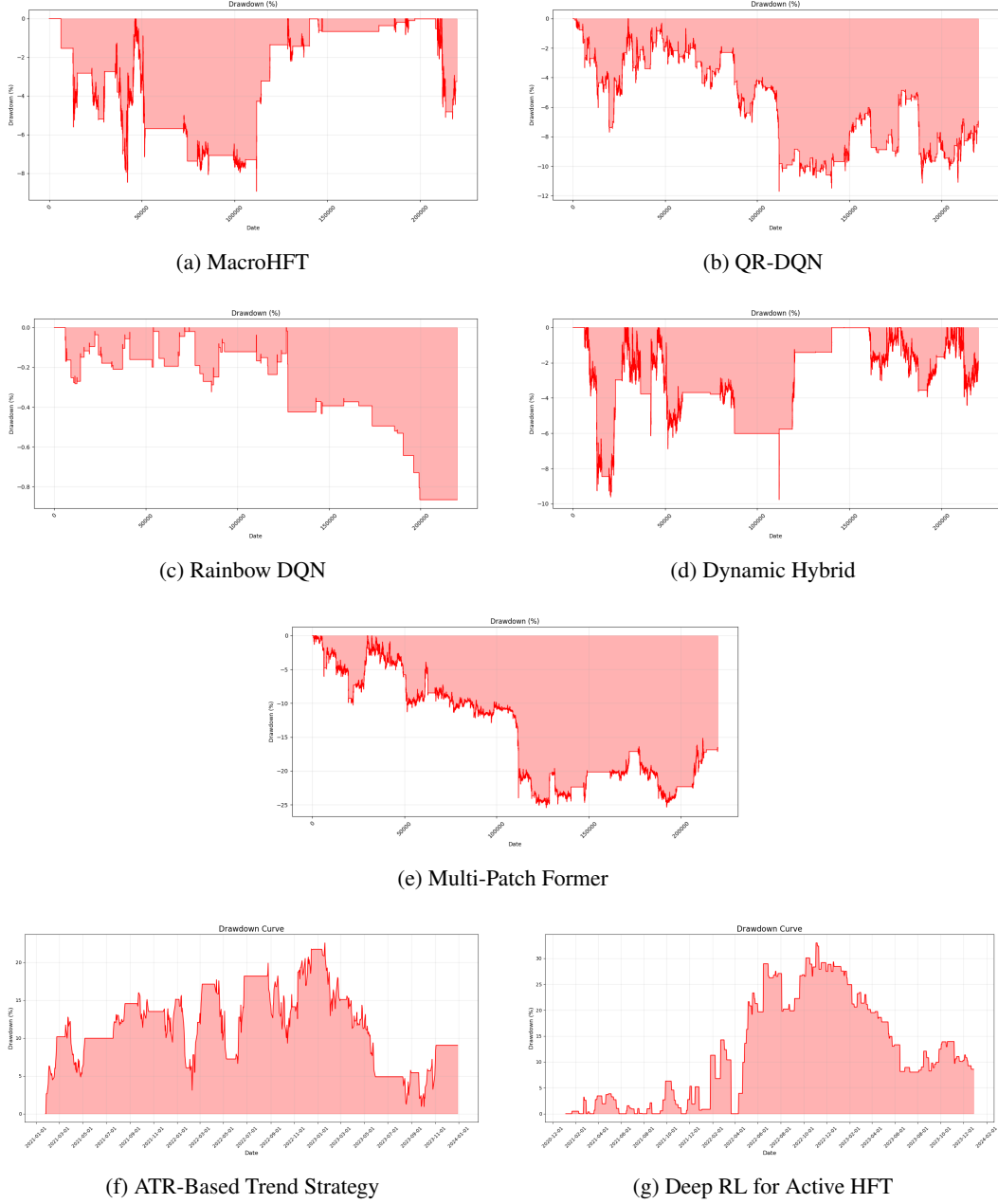
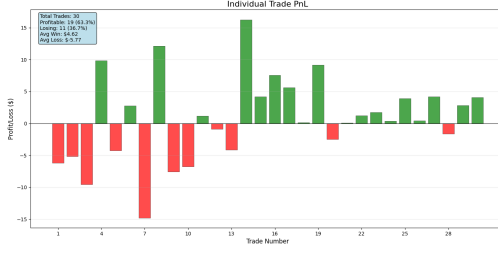


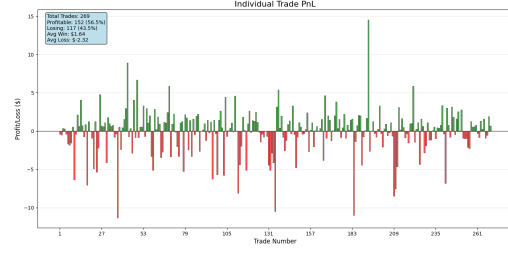
Figure 5: Maximum drawdown comparison across seven algorithms.

#### 6.2.4 Trade PnL Distribution

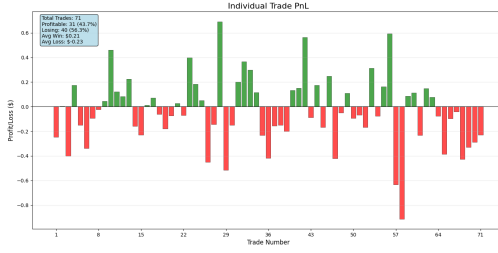
The profit and loss distribution of individual trades helps illustrate the skewness and volatility in trade outcomes.



(a) MacroHFT



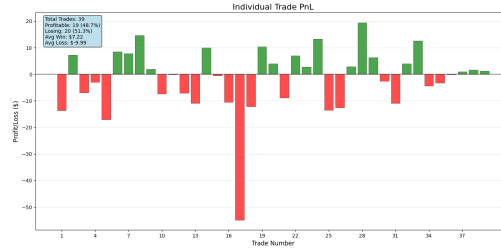
(b) QR-DQN



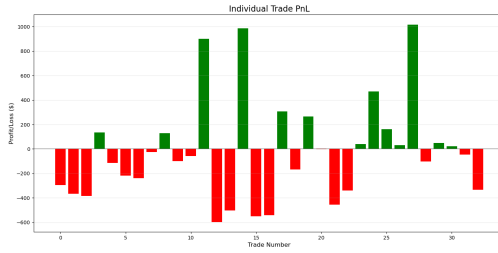
(c) Rainbow DQN



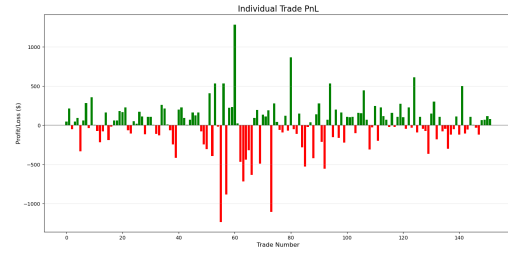
(d) Dynamic Hybrid



(e) Multi-Patch Former



(f) ATR-Based Trend Strategy



(g) Deep RL for Active HFT

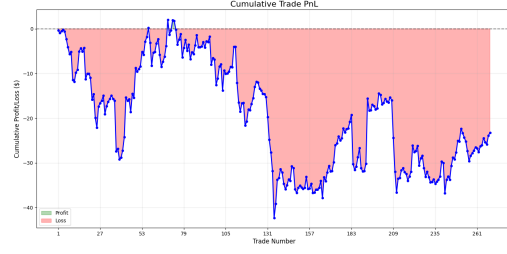
Figure 6: Trade-level P&L comparison across seven algorithms. Each chart illustrates the profit or loss from individual trades over time.

## 6.2.5 Cumulative Trade PnL

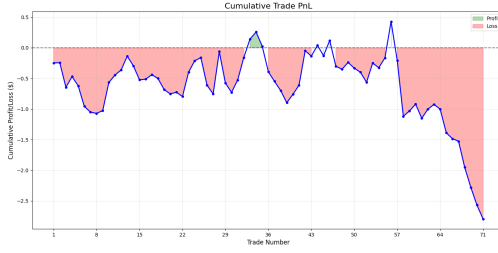
The cumulative trade-level profit and loss curve reveals how returns compound over successive trades, highlighting consistency and trend in performance.



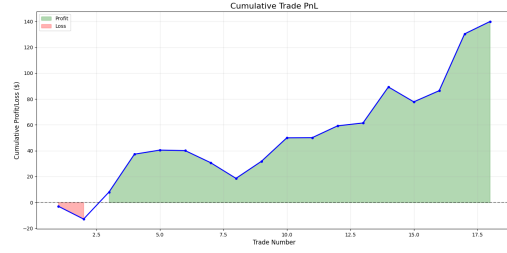
(a) MacroHFT



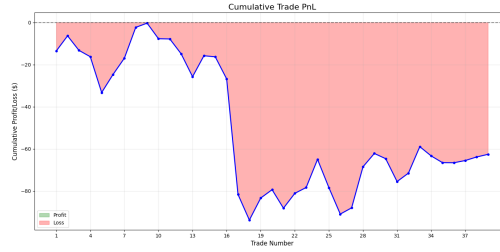
(b) QR-DQN



(c) Rainbow DQN



(d) Dynamic Hybrid

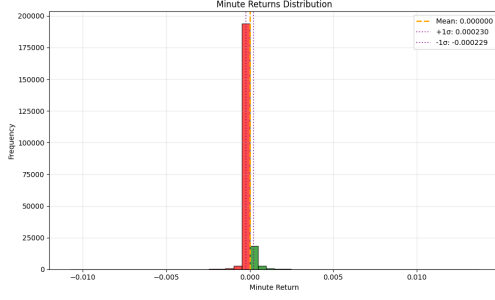


(e) Multi-Patch Former

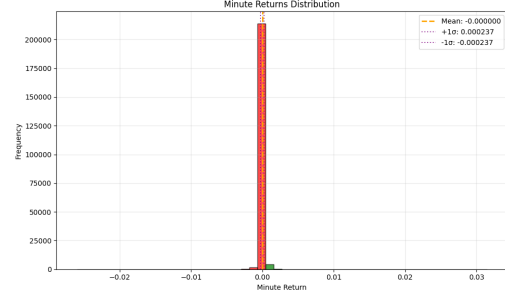
Figure 7: Cumulative trade-level P&L across five algorithms. The curves show how profits or losses evolve across sequential trades.

## 6.2.6 Minute Return Distribution

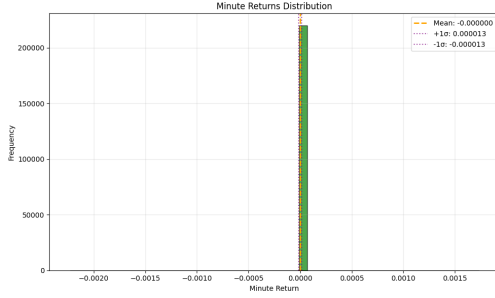
The distribution of per-minute returns reveals the volatility profile and asymmetry in short-term performance for each strategy.



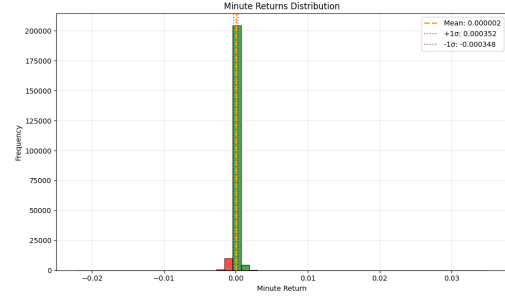
(a) MacroHFT



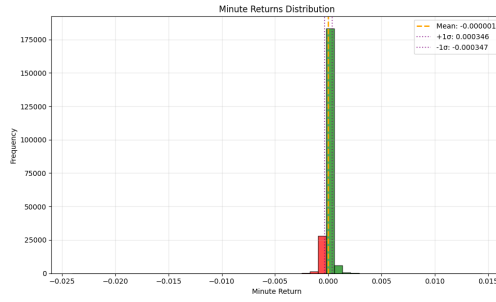
(b) QR-DQN



(c) Rainbow DQN



(d) Dynamic Hybrid



(e) Multi-Patch Former

Figure 8: Distribution of per-minute returns across five algorithms. Wider or skewed distributions indicate higher volatility or directional bias.

## 7 Conclusion

This study comprehensively investigated various reinforcement learning and sequence modeling approaches for high-frequency trading in volatile cryptocurrency markets, culminating in significant theoretical contributions and practical insights for adaptive trading system design. Through systematic experimentation with hierarchical agents, distributional reinforcement learning variants, and transformer-based architectures, we discovered that intelligent agent coordination mechanisms represent the key to achieving robust performance across diverse market regimes.

### Principal Research Findings

Our experimental evaluation conclusively demonstrates Dynamic Hybrid as the superior algorithmic approach, achieving exceptional performance that validates adaptive coordination mechanisms. Dynamic Hybrid delivered a remarkable 38.66% total return with a Sharpe ratio of 3.643, substantially outperforming all alternatives and establishing a new benchmark for intelligent trading systems. Combined with a Sortino ratio of 2.9678, profit factor of 2.0134, and win rate of 60.67%, these metrics demonstrate robust profit generation with effective downside risk management across diverse market conditions.

This performance superiority becomes particularly striking when compared with traditional approaches. While baseline MacroHFT achieved 0.6% return and Rainbow DQN managed 0.73%, Dynamic Hybrid’s results represent a quantum leap in effectiveness—more than a 50-fold improvement that validates our central hypothesis regarding market regime-dependent coordination mechanisms.

The distributional reinforcement learning approaches revealed instructive mixed results. Rainbow DQN demonstrated moderate effectiveness (0.73% return, 1.3288 Sharpe ratio), validating multi-technique integration benefits. However, QR-DQN’s negative performance highlights that distributional learning alone, without adaptive coordination, proves insufficient for volatile cryptocurrency markets, emphasizing that coordination methodology matters as much as uncertainty quantification.

Although theoretically well-suited for modeling temporal dependencies in financial time series, the Multi-Patch Former underperformed relative to other strategies in our experiments. This can be attributed to several practical challenges. The model’s performance is highly dependent on the choice of input sequence length, which requires fine-tuning, and without optimal configuration, key temporal patterns may be missed or misrepresented. The original implementation uses relatively few training epochs, and due to time constraints, we were unable to significantly increase the number of epochs in our training, potentially preventing the model from reaching full convergence. Additionally, sequence models such as the Multi-Patch Former are designed to extract trend-based patterns from time series data, but in highly volatile environments like cryptocurrency markets, short-term noise often dominates, reducing the effectiveness of models that rely on stable sequential structure.

### **Theoretical Contributions and Methodological Innovations**

This research introduces significant theoretical contributions at the intersection of reinforcement learning and financial applications. The Dynamic Hybrid framework represents a paradigm shift from static agent coordination to adaptive multi-agent collaboration, where coordination itself becomes a learned, context-dependent strategy. This innovation addresses fundamental ensemble method limitations by recognizing that different market conditions require fundamentally different decision-making paradigms—consensus-building during stable periods versus decisive expert selection during volatile transitions.

Our hierarchical architecture with conditional adaptation mechanisms provides a reusable framework extending beyond financial trading to any complex environment requiring adaptive decision-making. The integration of episodic memory with distributional reinforcement learning creates a novel experience-based learning approach that captures both distributional return characteristics and contextual market state similarities, enabling sophisticated decision-making that considers immediate distributional information alongside historical pattern recognition.

### **Practical Implications and Commercial Viability**

Dynamic Hybrid’s superior performance provides clear empirical guidance for practical trading system development. Financial institutions can leverage these findings to design adaptive trading systems that automatically adjust decision-making paradigms based on real-time market analysis. The demonstrated commercial viability metrics justify production implementation where capital preservation and consistent profitability are paramount.

The transformative potential of adaptive coordination mechanisms suggests that future competitive advantage in algorithmic trading lies not in sophisticated individual algorithms alone, but in intelligent frameworks that dynamically orchestrate multiple specialized approaches based on market conditions. This represents a fundamental shift in how we conceptualize trading system architecture.

### **Limitations and Future Research Directions**

Despite strong performance, Dynamic Hybrid faces limitations requiring acknowledgment. The approach relies heavily on accurate market regime identification, potentially challenging during unprecedented conditions or rapid transitions. Computational complexity may present scalability challenges for microsecond-latency applications.

Future research opportunities include enhanced market regime identification incorporating alternative data sources (news sentiment, social media, macroeconomic signals), multi-timescale coordination frameworks spanning millisecond execution to long-term allocation, extension to other fi-

nancial instruments and market structures, and advanced uncertainty quantification through modern Bayesian deep learning techniques.

### Concluding Remarks

This study demonstrates that sophisticated agent coordination mechanisms, particularly those adaptively balancing collective intelligence with specialized expertise, represent the frontier of algorithmic trading system design. Dynamic Hybrid’s superior performance validates context-aware, adaptive approaches over static ensemble methods, suggesting that algorithmic trading’s future lies in intelligent frameworks dynamically orchestrating multiple specialized approaches rather than developing increasingly sophisticated individual algorithms.

The broader implications extend beyond financial applications to any domain requiring adaptive decision-making under uncertainty. Dynamic Hybrid principles provide a template for intelligent systems that automatically adjust coordination strategies based on environmental conditions, representing significant advancement in multi-agent reinforcement learning for complex, time-varying environments.

Through systematic investigation of diverse algorithmic approaches, this research contributes practical trading system development tools and theoretical insights advancing our understanding of adaptive coordination mechanisms in complex financial environments. The demonstrated effectiveness establishes a strong foundation for future intelligent financial systems research.

## 8 Contributions

To ensure efficient collaboration and smooth progress, we have divided the project into three main areas: literature review and methodology design, data processing and environment implementation, and experimentation and performance evaluation. Below are the expected contributions of each team member:

- **Hung-Ting Hsieh(9/20):**

1. **Algorithm Development and Implementation:** Coordinated the research direction and conducted comprehensive development, implementation, and systematic testing of various reinforcement learning algorithms. Adapted and implemented Rainbow DQN and QR-DQN within the existing hierarchical framework, and developed the innovative Dynamic Hybrid coordination mechanism that became the key contribution of this research. Optimized sophisticated algorithms and focused on integrating distributional reinforcement learning techniques with adaptive coordination mechanisms for improved trading performance.
2. **Architecture Adaptation and Enhancement:** Modified and enhanced existing simulation environments to support advanced reinforcement learning algorithms and coordination mechanisms. Implemented adaptive switching mechanisms and coordination modules for dynamic strategy selection.
3. **Theoretical Analysis and Knowledge Integration:** Coordinated comprehensive theoretical analysis by studying advanced RL theory papers and collaborating with team members to create educational content. This theoretical foundation informed the development of the Dynamic Hybrid methodology and provided insights for connecting theory with practical applications.
4. **Literature Review and Research Coordination:** Performed literature survey on reinforcement learning algorithms and multi-agent coordination techniques. Analyzed baseline papers and coordinated research activities to identify implementation opportunities and methodological improvements.
5. **Report and Presentation Contributions:** Served as the primary co-author of the term project proposal, technical report, and research poster, with emphasis on methodology sections and research insights. Contributed to research documentation and presentation materials.

- **Chun-Yu Lin(1/20):**

1. **Literature Review and Paper Analysis:** Conducted systematic reading and analysis of relevant research papers to support the theoretical foundation of the project.

2. **Theoretical Deep Dive Collaboration:** Collaborated in the comprehensive theoretical analysis by contributing to the study of advanced RL theory papers and assisting in the creation of educational content for the deep dive video presentation.
- **Chien-Cheng Chu(10/20):**
    1. **Baseline Algorithm Implementation:** Developed and benchmarked both deep reinforcement learning (DRL) algorithms and a traditional ATR-based trend-following strategy for comparative analysis.
    2. **Network and Algorithm Enhancements:** Extended the baseline DDQN architecture by incorporating custom modifications, including the integration of the Multi-Patch Former for sequence modeling in high-frequency financial data.
    3. **Environment Adaptation:** Modified the simulation environment to support time-series structured state representations, enabling effective processing by sequence-based models.
    4. **Performance Evaluation:** Conducted comprehensive performance analysis across all strategies, including metric computation, visualizations (e.g., cumulative return, drawdown, trade PnL), and comparative performance tables.
    5. **Research and Literature Review:** Performed in-depth literature survey and paper reading on state-of-the-art reinforcement learning algorithms and financial time series modeling techniques.
    6. **Report and Presentation Contributions:** Served as the primary author of the term project proposal, technical report, and research poster, and delivered the poster presentation.



## References

- Antonio Briola, Jeremy Turiel, Riccardo Marcaccioli, Alvaro Cauderan, and Tomaso Aste. Deep reinforcement learning for active high frequency trading. *arXiv preprint arXiv:2101.07107*, 2023. URL <https://arxiv.org/abs/2101.07107>.
- Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. *arXiv preprint arXiv:1710.02298*, 2017. doi: 10.48550/arXiv.1710.02298. URL <https://arxiv.org/abs/1710.02298>.
- Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. *arXiv preprint arXiv:1710.10044*, 2017. doi: 10.48550/arXiv.1710.10044. URL <https://arxiv.org/abs/1710.10044>.
- Chuang Zong, Chaojie Wang, Molei Qin, Lei Feng, Xinrun Wang, and Bo An. Macrohft: Memory augmented context-aware reinforcement learning on high frequency trading. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. ACM, 2024. doi: 10.1145/3637528.3672064. URL <https://doi.org/10.1145/3637528.3672064>.
- Guanghe Cao, Yitian Zhang, Qi Lou, and Gaikang Wang. Optimization of high-frequency trading strategies using deep reinforcement learning. *Journal of Artificial Intelligence General Science (JAIGS)*, 6(1), 2024. doi: 10.60087. URL <https://ojs.boulibrary.com/index.php/JAIGS>.
- Jie Zou, Jiashu Lou, Baohua Wang, and Sixue Liu. A novel deep reinforcement learning based automated stock trading system using cascaded lstm networks. *arXiv preprint arXiv:2212.02721*, 2022. URL <https://arxiv.org/abs/2212.02721>.
- Yang Li, Wanshan Zheng, and Zibin Zheng. Deep robust reinforcement learning for practical algorithmic trading. *IEEE Access*, 7, 2019. doi: 10.1109/ACCESS.2019.2932789. URL <https://ieeexplore.ieee.org/document/8786132>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Yue Wang, Xiaoyu Chen, Mengyuan Zhang, and Liang Zhao. MPFormer: Multi-patch transformer for multi-variate time series anomaly detection with contrastive learning. *IEEE Internet of Things Journal*, 2024. doi: 10.1109/IIOT.2024.3443910. URL <https://ieeexplore.ieee.org/document/10460114>.